Solving ODE Using PINN

What is PINN?

A **Physics-Informed Neural Network (PINN)** is a modern approach in scientific machine learning that blends traditional physics with deep learning. Instead of relying purely on data, a PINN incorporates known physical laws—typically expressed as differential equations—directly into the learning process of a neural network. This makes it especially powerful for solving scientific and engineering problems where data is limited but governing equations are well understood.

Conceptual Foundation

In classical machine learning, a neural network is trained to map inputs to outputs using large datasets. However, many real-world systems—such as fluid flow, heat transfer, or population dynamics—are governed by **mathematical equations** rather than just data. A PINN leverages this by embedding these equations into the training process.

For example, if a system is described by a differential equation, instead of solving it numerically using traditional methods (like finite difference or finite element methods), a PINN approximates the solution using a neural network while ensuring that the solution satisfies the equation.

How It Works

At the core of a PINN is a neural network that represents the unknown solution function. Suppose we want to approximate a function (u(x, t)), which might represent temperature, velocity, or some physical quantity. The neural network takes inputs like space (x) and time (t), and outputs a predicted value (u).

What makes PINNs unique is how they are trained. The training process involves minimizing a **loss function** that has multiple components:

1. **Data Loss** – This ensures the network fits any available observed data.
2. **Physics Loss** – This enforces that the neural network's output satisfies the governing physical equation.

Using **automatic differentiation** (a feature available in frameworks like PyTorch, TensorFlow, and JAX), the network can compute derivatives of its output with respect to inputs. These derivatives are then substituted into the differential equation to measure how well the predicted solution satisfies the physics.

The model is trained so that both the data error and the physics error are minimized simultaneously.

Types of Problems PINNs Address

PINNs are highly versatile and can be applied to:

- **Ordinary Differential Equations (ODEs)**: Simple time-dependent systems
- **Partial Differential Equations (PDEs)**: Heat equation, wave equation, Navier-Stokes equations
- **Inverse Problems**: Estimating unknown parameters in physical systems
- **Data-driven modeling**: Where only partial observations are available

Advantages

One of the biggest strengths of PINNs is their ability to work with **limited or noisy data**, since the physics itself guides the learning process. They are also mesh-free, meaning they do not require discretization of the domain like traditional numerical methods.

Additionally, PINNs unify data and theory, allowing them to solve problems that are difficult for either approach alone.

Challenges

Despite their advantages, PINNs come with some limitations:

- Training can be computationally expensive
- Balancing different components of the loss function can be tricky

- They may struggle with highly complex or chaotic systems

Intuitive Understanding

You can think of a PINN as a student learning physics. Instead of just memorizing examples (data), the student also understands the laws of physics (equations). This combination allows the student to generalize better and make accurate predictions even with limited information.

Step-by-Step Explanation of PINN ODE Solver Code

1. Importing Libraries

```
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
```

2.Device Configuration

- torch → PyTorch (used for neural networks / PINNs)
- torch.nn → neural network module
- numpy → numerical computations
- matplotlib.pyplot → plotting graphs

```
# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

The device configuration is used to dynamically select between CPU and GPU execution. If a CUDA-enabled GPU is available, computations are performed on the GPU for faster processing; otherwise, the CPU is used. This ensures optimal performance and compatibility across different hardware environments.

3.Neural Network Definition (PINN Model Definition)

```python
# Neural Network
class PINN(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(1, 50),
            nn.Tanh(),
            nn.Linear(50, 50),
            nn.Tanh(),
            nn.Linear(50, 1)
        )

    def forward(self, x):
        return torch.exp(-x**2) * self.net(x)

model = PINN().to(device)
```

In this step, a Physics-Informed Neural Network (PINN) model is defined using a fully connected feedforward architecture. The network consists of multiple hidden layers with Tanh activation functions to ensure smooth function approximation. The forward pass incorporates a trial solution by multiplying the neural network output with an exponential function, which helps enforce boundary conditions implicitly

```
Input (x)
   ↓
Linear Layer (1 → 50)
   ↓
Tanh Activation
   ↓
Linear Layer (50 → 50)
   ↓
Tanh Activation
   ↓
Linear Layer (50 → 1)
   ↓
Multiply with exp(-x²)
   ↓
Final Output
```

4. Training Data Preparation

```python
# Training data
x1 = torch.linspace(0, 2, 1000)
x2 = torch.linspace(2, 4, 3000)  # more focus here
x = torch.cat([x1, x2]).reshape(-1,1).to(device)
x = (x - 2.0) / 2.0   # normalize to [-1,1]
x = x.to(device)
x.requires_grad = True
```

In this step, training data is generated by sampling collocation points from the problem domain. A higher density of points is allocated in regions of interest to improve solution accuracy. The input data is normalized to enhance training stability and is configured to require gradients, enabling automatic differentiation for enforcing the governing differential equations in the PINN framework.

```
Domain [0,4]
   ↓
Split into regions
   ↓
Dense sampling in important region
   ↓
Merge points
   ↓
Reshape for NN input
   ↓
Normalize to [-1,1]
   ↓
Enable gradients
```

5.Boundary Condition Definition

```
# Boundary points (helps stabilize edges)
x_bc = torch.tensor([[0.0], [4.0]]).to(device)
```

In this step, boundary points are defined at the edges of the computational domain. These points are used to enforce boundary conditions during training, ensuring that the neural network solution satisfies the physical constraints at the domain boundaries.

6. Optimization and Loss Function Computation

```
# Optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

```
optimizer_lbfgs = torch.optim.LBFGS(model.parameters(), lr=0.1)

def closure():
    optimizer_lbfgs.zero_grad()
    y = model(x)
    dy_dx = torch.autograd.grad(y, x, torch.ones_like(y), create_graph=True)[0]
    loss = torch.mean((dy_dx + 2*x*y)**2) + (model(torch.tensor([[0.0]]).to(device)) - 1)**2
    loss.backward()
    return loss

optimizer_lbfgs.step(closure)
```

```
tensor([[1.3256]], grad_fn=<AddBackward0>)
```

In this step, the model is trained by minimizing a physics-informed loss function. The loss consists of two components: the residual of the governing differential equation and the boundary condition error. Automatic differentiation is used to compute derivatives required for enforcing the physical laws. The optimization is performed using a combination of Adam and LBFGS optimizers to ensure both fast convergence and high accuracy.

```
Input x
   ↓
Model Prediction y
   ↓
Compute dy/dx (autograd)
   ↓
Compute Physics Loss
   ↓
Compute Boundary Loss
   ↓
Total Loss
   ↓
Backpropagation
   ↓
Optimizer Update (LBFGS)
```

7. Model Training

```
    y = model(x)

    # First derivative
    dy_dx = torch.autograd.grad(
        y, x,
        grad_outputs=torch.ones_like(y),
        create_graph=True
    )[0]

    # ODE loss: dy/dx + 2xy = 0
    ode_loss = torch.mean((dy_dx + 2*x*y)**2)

    # Initial condition y(0)=1
    x0 = torch.tensor([[0.0]]).to(device)
    y0 = model(x0)
    ic_loss = (y0 - 1)**2

    loss = 10 * ode_loss + ic_loss

    loss.backward()
    optimizer.step()

    if epoch % 500 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item()}")
```

```
Epoch 0, Loss: 0.07185637205839157
Epoch 500, Loss: 1.7096775764002814e-06
Epoch 1000, Loss: 1.4914175153535325e-05
Epoch 1500, Loss: 9.959909448298276e-07
Epoch 2000, Loss: 8.357818614967982e-07
Epoch 2500, Loss: 1.2028042419842677e-06
```

In this step, the PINN model is trained using an iterative optimization process. For each epoch, the model predicts outputs, computes derivatives using automatic differentiation, and evaluates the residual of the governing differential equation. The total loss is formed by combining the physics-based loss and the initial condition loss. Backpropagation is used to compute gradients, and the optimizer updates the model parameters to minimize the loss. The decreasing loss over epochs indicates convergence toward the true solution.

```
Start Training
   ↓
Forward Pass (model(x))
   ↓
Compute dy/dx
   ↓
Compute ODE Loss
   ↓
Compute IC Loss
   ↓
Total Loss
   ↓
Backpropagation
   ↓
Update Weights
   ↓
Repeat (epochs)
```

8.Model Testing

```
# Testing
x_test = torch.linspace(0, 4, 1000).reshape(-1,1).to(device)
y_pred = model(x_test).detach().cpu().numpy()
```

In this step, the trained PINN model is evaluated by generating test inputs across the domain. The model predicts the corresponding outputs, which represent the approximate solution of the differential equation. The predictions are detached from the computation graph and converted into NumPy arrays for further analysis and visualization.

9.Model Evaluation and Visualization

```
# Exact solution
x_np = x_test.cpu().numpy()
y_exact = np.exp(-x_np**2)
```

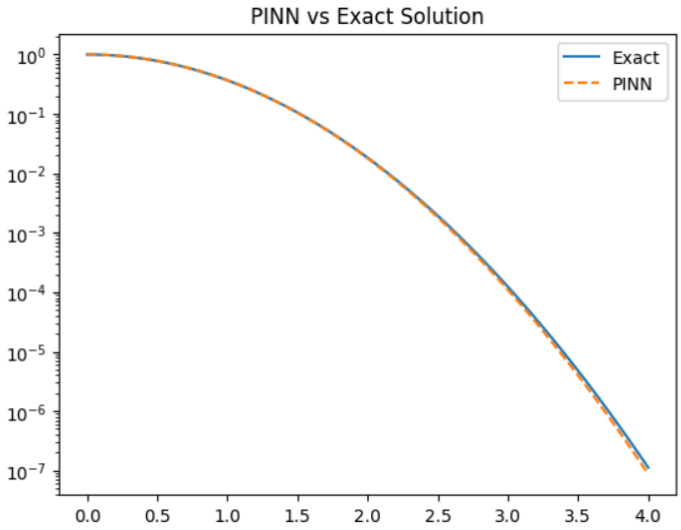
```
# Plot
plt.plot(x_np, y_exact, label="Exact")
plt.plot(x_np, y_pred, '--', label="PINN")
plt.legend()
plt.yscale("log")
plt.title("PINN vs Exact Solution")
plt.show()
```

In this step, the predicted solution obtained from the PINN model is compared with the exact analytical solution. The exact solution is computed using the known mathematical expression and plotted alongside the model predictions. A logarithmic scale is used to highlight small differences between the two curves. The close agreement between the predicted and exact solutions demonstrates the effectiveness of the PINN in solving the differential equation.

Final workflow

```
Input Domain
     ↓
Neural Network (PINN)
     ↓
Physics-based Loss
     ↓
Training (Optimization)
     ↓
Prediction
     ↓
Exact Solution
     ↓
Comparison Plot ✅
```

Output



Summary

A Physics-Informed Neural Network is a powerful tool that integrates deep learning with physical laws to solve complex scientific problems. By embedding equations into the training process, PINNs ensure that predictions are not only accurate but also physically meaningful.